

BRAT EAT — MASTER CTO / ENGINEERING DOSSIER

VERSION

V1 OFFICIAL ENGINEERING SOURCE OF TRUTH

1. EXECUTIVE SUMMARY

BRAT EAT is not a classic food ordering application.

BRAT EAT is:

- a realtime event ordering platform,
- optimized for stadiums, arenas, festivals and corporate environments,
- powered by Flaix AI flow orchestration,
- designed for ultra-fast click & collect,
- capable of dynamically adapting to crowd density and operational pressure.

The primary mission:

Reduce waiting time, optimize crowd movement and create the fastest premium event ordering experience possible.

The application must feel:

- premium,
- instant,
- intelligent,
- fluid,
- emotionally satisfying.

The product positioning is closer to:

- Uber Eats realtime UX,
 - Burger King mobile simplicity,
 - event operational software,
 - AI-powered venue orchestration.
-

2. CORE PRODUCT PHILOSOPHY

The user must immediately feel:

- "Everything is fast."
- "Everything is simple."
- "I will save time."
- "This venue is modern."
- "The app understands the rush."

Product philosophy:

Speed > complexity.

Operational reliability > visual effects.

Realtime synchronization > feature quantity.

The most important UX objective:

The user orders during a rush and receives the order significantly faster than traditional queues.

3. V1 STRATEGY

V1 includes:

- Click & collect
- Corporate delivery mode
- Smart pickup slots
- Realtime dashboards
- Public ready screens
- Push notifications
- Favorites
- Smart reorder
- Loyalty system
- Dynamic CMS
- Feature flags
- Flaix integration
- Operator dashboards
- Multi-organization support
- Multi-event support

V1 excludes:

- Microservices
- Advanced fraud engine

- Offline mode
 - Complex split kitchen preparation
 - Advanced RGPD automation
 - Advanced accounting engine
-

4. MAIN DIFFERENTIATOR

Flaix

Flaix is the strategic moat.

No competitor currently orchestrates:

- crowd flow,
- smart slotting,
- operational pressure,
- queue intelligence,
- dynamic order pacing,
- realtime pickup optimization,

at this level.

Flaix controls:

- pickup slot optimization,
- rush scoring,
- dynamic throttling,
- queue balancing,
- intelligent recommendations,
- preparation pacing,
- flow management.

Flaix is the source of truth.

5. TARGET ENVIRONMENTS

Main targets

- Stadiums
- Arenas
- Sports clubs
- Festivals
- Corporate campuses
- Event venues

Example deployment environments

- Hockey arena
 - Football stadium
 - Corporate office tower
 - Multi-vendor food court
 - Festival zone
-

6. TARGET SCALE

V1 objectives

- 50,000 simultaneous users
- 200 suppliers simultaneously
- 100+ orders during rush
- Multi-event support
- Realtime synchronization

Long-term objectives

- European deployment
 - Stadium standard platform
 - 100,000+ active users
 - Multi-country cloud deployment
-

7. CORE UX PRINCIPLES

Mandatory UX requirements

Mobile-first

Everything optimized for mobile first.

Instant navigation

Native-feeling navigation mandatory.

Minimal friction

Fewest possible steps.

Realtime feeling

The app must always feel alive.

Premium loading states

Use skeleton shimmer loaders.

Background preloading

Preload:

- images,
- menus,
- categories,
- branding,
- products.

Quick emotional reward

User must feel rewarded quickly.

8. TECH STACK

Mobile

Mandatory stack

- React Native
- Native Stack Navigation
- Zustand
- TanStack Query
- TypeScript strict mode

Recommendation

Use React Native CLI.

Reason:

- more scalable,
 - more native control,
 - better realtime capabilities,
 - better future enterprise scaling.
-

Backend

Mandatory stack

- NestJS
- PostgreSQL
- Redis
- WebSockets
- Stripe
- S3-compatible storage

Architecture strategy

Monolith first.

Reason:

- Faster development
- Easier maintenance
- Lower infrastructure complexity
- Easier debugging
- Better V1 velocity

NO microservices in V1.

Infrastructure

Required

- CI/CD GitHub
- TestFlight
- Play Beta
- Sentry monitoring
- Docker
- Cloud deployment

Future

- Multi-region deployment
 - Europe + USA scaling
-

9. ARCHITECTURE RULES

Mandatory rules

- Feature-based architecture
- Strict separation:
 - UI
 - business logic
 - API layer
- No duplicated logic
- Reusable components mandatory
- Feature flags mandatory
- Realtime-first architecture
- Multi-tenant ready
- Event-driven state transitions

Folder structure example

```
/features/auth  
/features/orders  
/features/products  
/features/events  
/features/dashboard  
/features/flaix  
/features/payments  
/features/notifications
```

10. STATE MANAGEMENT

Global state

Use Zustand.

Reason:

- simpler than Redux,
- cleaner,
- scalable enough,
- excellent developer velocity.

Server cache

Use TanStack Query.

Cache:

- menus,
 - products,
 - categories,
 - dashboards,
 - orders.
-

11. NAVIGATION

Mandatory

- Native Stack Navigation
- Instant transitions
- Prefetch screens
- Smooth animations

Changing:

- event,
- venue,
- organization,

must feel instant.

12. DESIGN SYSTEM

Mandatory components

- Button
- ProductCard
- DashboardCard
- Modal
- BottomSheet
- NotificationBanner
- SlotSelector
- Timeline
- SkeletonLoader
- StatusBadge
- RealtimeIndicator

UX style

Closer to:

- Uber Eats,

- Burger King,
 - Apple fluidity.
-

13. PRODUCT FLOW

First app launch

User sees:

- Premium onboarding
- Nearby events
- Venue discovery
- Quick explanation
- Event highlights

Guest browsing allowed.

Ordering requires account creation.

Authentication

Supported

- Email/password
- Apple Sign In
- Google Sign In

Future

- Enterprise SSO
-

Ordering flow

Main objective

Ultra-fast checkout.

Mandatory

- One-step checkout
- Smart slot selection
- Auto “as soon as possible” logic

- Dynamic pickup timing
-

14. SLOT ENGINE

Core functionality

Pickup slots are strategic.

Slots can be:

- dynamic,
- manual,
- event-based,
- supplier-based,
- rush-based.

Example

- Pickup ASAP
- Pickup at halftime
- Pickup at arrival
- Pickup at custom slot

Flaix controls

- slot optimization,
 - queue balancing,
 - overload prevention,
 - rush pacing.
-

15. EVENT SYSTEM

Events support

An event can contain:

- multiple suppliers,
- multiple pickup points,
- multiple dashboards,
- multiple public screens.

Example pickup points

- North stand
- VIP lounge

- Rooftop
 - Arena parvis
 - Floor 2
-

16. MULTI-VENDOR SYSTEM

One cart can contain:

- food supplier
- drinks supplier
- dessert supplier

inside same checkout.

Mandatory architecture consideration

Cart system must support:

- multiple suppliers,
 - multiple preparation zones,
 - multiple pickup logic.
-

17. PRODUCT SYSTEM

Product features

Products support:

- categories,
- schedules,
- start/end dates,
- event visibility,
- dynamic recommendations,
- upsells,
- cross-sell,
- loyalty logic.

Recommendations

Recommendations appear:

- after add to cart,
 - during checkout,
 - during rush optimization.
-

18. DASHBOARD SYSTEM

Critical system

Dashboard reliability is mission-critical.

Dashboard types

- New orders
- Immediate preparation
- Rush mode
- Ready orders
- Recovered orders
- Public displays
- Global summary

Dashboard features

- Fullscreen kiosk mode
- Realtime updates
- Sound alerts
- Product summaries
- Dynamic filters
- Conditional displays
- Multi-zone logic
- Rush simplification mode

Public screens

Public screens display:

- order number,
- preparation status,
- ready status,
- grouped product summaries.

NO complex customer information.

19. REALTIME SYSTEM

Mandatory realtime events

- New order
- Order ready
- Order recovered
- Rush status changed
- Product unavailable

- Slot updated
- Queue overloaded

User notifications

When order becomes ready:

- popup,
 - vibration,
 - optional sound,
 - strong visual feedback.
-

20. RUSH MODE

Mandatory

Operator rush mode.

During rush:

- simplified UI,
- larger interaction zones,
- faster transitions,
- reduced distractions.

Flaix can:

- slow incoming orders,
 - modify slots,
 - pause ordering,
 - optimize queue.
-

21. CMS ENGINE

Core philosophy

The CMS is strategic.

Organizations can:

- activate modules,
- reorder sections,
- configure categories,
- upload banners,
- change wording,

- manage schedules,
- configure dashboards.

Custom designs

Premium custom layouts: ONLY BRAT EAT internal team can create.

Organizations use:

- locked templates,
 - configurable blocks,
 - controlled personalization.
-

22. PAGE BUILDER

Hybrid strategy

Use:

- locked templates,
- configurable sections,
- drag-and-drop ordering.

Mandatory features

- Draft
 - Preview
 - Publish
 - Rollback versions
 - Scheduled publishing
 - iPhone preview
 - Android preview
 - Tablet preview
-

23. FEATURE FLAGS

Mandatory

Feature flags exist:

- globally,
- by organization,
- by event.

Examples

Enable:

- wallet,
- loyalty,
- push marketing,
- corporate delivery,
- enterprise mode,
- merchandising,
- Flaix features.

WITHOUT redeployment.

24. CORPORATE MODE

Corporate support

Corporate mode supports:

- office delivery,
- floor number,
- building number,
- custom slots,
- enterprise catalogs,
- private pricing,
- badge integrations.

Future

- SSO
 - Microsoft login
 - Workspace integration
-

25. PAYMENT SYSTEM

V1 payment

- Stripe
- Retry payment
- Save cart on failure
- Badge integrations possible

Failure behavior

If payment fails:

- clear error message,
 - no order creation,
 - retry immediately available.
-

26. STOCK ENGINE

Stock rules

Stock is realtime.

Stock managed:

- by pickup point,
- by supplier.

If stock reaches zero:

- product becomes unavailable.
-

27. PUSH NOTIFICATIONS

Notifications center mandatory

Notifications:

- orders,
- promotions,
- events,
- rush updates.

Geolocalized push support

Example: "You are close to pickup zone."

Future

Behavior-based automation.

28. ANALYTICS ENGINE

Basic V1 KPIs

- orders count
- revenue
- rush periods
- top products
- supplier performance
- slot usage

Future advanced analytics

- heatmaps
 - behavior analysis
 - predictive rush
 - comparative events
-

29. LOYALTY SYSTEM

V1 includes

- loyalty system
- favorites
- smart reorder
- order history

Example

"Reorder your usual menu."

30. PERFORMANCE REQUIREMENTS

Critical requirement

Homepage must load extremely fast.

Mandatory

- image lazy loading
- cache optimization
- background prefetching
- websocket optimization
- minimal blocking renders

31. QA STRATEGY

Mandatory

Heavy real terrain testing.

Priority

Rush testing.

Beta strategy

Closed beta before public launch.

32. SECURITY

V1 strategy

Keep security simple but solid.

Mandatory

- JWT auth
- secure APIs
- rate limiting
- payment isolation
- permission system

Fraud engine postponed.

33. ADMIN ROLES

Required roles

- Super Admin
- Organization Admin
- Manager
- Operator
- Marketing

Permissions system mandatory.

34. CLAUDE CODE ROLE

Claude Code responsibilities

Claude Code is the MAIN ARCHITECT.

Claude must:

- design scalable architecture,
- generate backend modules,
- maintain engineering consistency,
- create documentation,
- explain decisions,
- generate technical summaries.

Mandatory Claude output after each task

Claude MUST generate:

TASK_SUMMARY .md

Containing:

- what was created,
- why it was created,
- architecture decisions,
- impacted modules,
- files created,
- files modified,
- next dependencies,
- warnings.

Claude coding rules

- NEVER generate entire app at once
 - Work module-by-module
 - Respect architecture rules
 - No duplicate logic
 - Strict TypeScript
 - Realtime safe code only
 - Keep V1 simple
 - No overengineering
-

35. CURSOR ROLE

Cursor responsibilities

Cursor is the DAILY IMPLEMENTATION TOOL.

Cursor handles:

- frontend iterations,
- UI refinement,
- component polishing,
- local debugging,
- rapid code editing,
- small refactors.

Cursor should NOT redesign architecture.

Cursor follows:

- ARCHITECTURE.md
 - AGENTS.md
 - DESIGN_SYSTEM.md
-

36. CODEX ROLE

Codex responsibilities

Codex handles:

- repetitive generation,
- parallel implementations,
- utility generation,
- test generation,
- repetitive CRUD generation,
- automation support.

Codex should NOT:

- redesign product logic,
 - modify architecture decisions.
-

37. DOCUMENTATION STRUCTURE

Mandatory root structure

```
/brain  
  PRODUCT_VISION.md  
  ARCHITECTURE.md  
  AGENTS.md  
  DESIGN_SYSTEM.md  
  ROADMAP.md  
  TASK_SUMMARY.md
```

brain folder is the official source of truth.

AI tools MUST read it before coding.

38. DEVELOPMENT STRATEGY

Official workflow

STEP 1

Generate architecture only.

STEP 2

Generate backend foundation.

STEP 3

Generate auth.

STEP 4

Generate organizations/events.

STEP 5

Generate products/categories.

STEP 6

Generate ordering engine.

STEP 7

Generate slot engine.

STEP 8

Connect Flaix.

STEP 9

Generate dashboards.

STEP 10

Generate public screens.

STEP 11

Generate CMS.

STEP 12

Optimization.

39. SPRINT PLANNING

Sprint 1

Foundation

- Monorepo
 - CI/CD
 - Docker
 - NestJS base
 - React Native base
 - Auth
 - Feature flags foundation
-

Sprint 2

Core entities

- Organizations
- Events
- Venues

- Products
 - Categories
 - Suppliers
-

Sprint 3

Ordering system

- Cart
 - Checkout
 - Slots
 - Orders
 - Stripe
 - Realtime engine
-

Sprint 4

Dashboards

- Operator dashboards
 - Public displays
 - Rush mode
 - Realtime updates
-

Sprint 5

CMS

- Templates
 - Sections
 - Scheduling
 - Publishing
 - Previews
-

Sprint 6

Flaix integration

- Rush scoring
- Slot intelligence
- Queue pacing

- Dynamic recommendations
-

Sprint 7

QA

- Terrain testing
 - Rush testing
 - Load testing
 - Stabilization
-

Sprint 8

Deployment

- TestFlight
 - Play Beta
 - Monitoring
 - Production deployment
-

40. ABSOLUTE PRIORITIES

The 3 things that MUST NEVER fail

1. Order transmission
 2. Dashboard realtime synchronization
 3. Pickup slot logic
-

41. MOST IMPORTANT SCREENS

Most important user screen

- Home
- Ordering flow

Most critical technical screen

- Dashboard realtime orders

First screen to develop

- Onboarding
-

42. COMPETITIVE ADVANTAGE

Why BRAT EAT wins

Because competitors:

- do not orchestrate venue flow,
- do not optimize rush dynamically,
- do not integrate intelligent slot pacing,
- do not operate like a stadium operating system.

BRAT EAT combines:

- realtime operations,
 - event ordering,
 - AI orchestration,
 - crowd optimization,
 - premium UX.
-

43. FINAL PRODUCT VISION

Long-term objective

BRAT EAT becomes:

The intelligent operating system for stadium ordering and crowd orchestration.

Flaix becomes:

- invisible intelligence layer,
 - standard orchestration engine,
 - realtime event optimization system.
-

44. FINAL ENGINEERING RULES

NON-NEGOTIABLE RULES

- NEVER break realtime order flow
- NEVER block UI during rush

- NEVER overengineer V1
 - ALWAYS optimize speed first
 - ALWAYS preserve dashboard stability
 - ALWAYS support feature flags
 - ALWAYS respect strict architecture separation
 - ALWAYS document decisions
 - ALWAYS produce TASK_SUMMARY.md after implementation
 - ALWAYS keep Flaix as source of truth
-

END OF MASTER DOSSIER